



Practica N°1.

Ejercicio 3.

Materia:

Bases de Datos

Profesor:

Gabriel Hurtado Avilés

Grupo:

3CV2

Presentan:

Ramirez González José Issac

Silva Anaya Ariana





1. Dato, Información y Base de Datos: Análisis Comparativo de Autores

En la ciencia de la computación, la comprensión de la jerarquía conceptual entre dato, información y base de datos constituye el pilar fundamental para el diseño de sistemas de información estructurados. Un dato representa la unidad mínima de representación simbólica (numérica, alfabética o algorítmica) carente de contexto intrínseco. La información surge al procesar, organizar y contextualizar un conjunto de datos, otorgándoles un significado comprensible para la toma de decisiones. Por su parte, una base de datos es la infraestructura lógica y física estructurada que almacena dichos datos organizados de forma persistente.

Definiciones Formales de Autores

Según Silberschatz, Korth y Sudarshan (2020), una base de datos se define como «una colección de datos interrelacionados y un conjunto de programas para acceder a dichos datos. El objetivo principal de una base de datos es proporcionar un entorno que sea a la vez conveniente y eficiente para ser utilizado en la extracción y almacenamiento de información» (p. 1). Su enfoque prioriza la eficiencia operacional y la abstracción del software de gestión.

Por otro lado, Elmasri y Navathe (2016) definen una base de datos como «una colección de datos lógicamente coherente con un significado implícito, diseñada, construida y poblada con datos para un propósito específico, representando aspectos del mundo real (minimundo)» (p. 4). Su enfoque enfatiza el modelado conceptual del mundo real y la semántica de la información.





Criterio de Comparación	Silberschatz, Korth y Sudarshan (2020)	Elmasri y Navathe (2016)
Enfoque Principal	Sistémico y operativo (eficiencia del software y acceso)	Semántico y conceptual (representación del minimundo)
Propósito del Sistema	Proporcionar un entorno conveniente para almacenamiento y extracción	Reflejar de manera fiel y coherente un propósito específico del negocio
Perspectiva de Datos	Conjunto de datos estructurados para ser manipulados por programas	Colección de datos lógicamente integrados con un significado propio

2. Características Fundamentales de una Base de Datos

Un sistema de base de datos profesional se distingue por un conjunto de características esenciales descritas por Date (2004) y Connolly & Begg (2015):

- Integración: Los archivos que componen la base de datos están unificados lógicamente, eliminando las fronteras entre aplicaciones aisladas.
- Persistencia: Los datos residen en medios de almacenamiento no volátiles de manera permanente, garantizando su existencia tras el cierre de aplicaciones o reinicios del sistema.
- Redundancia Controlada: Se minimiza la duplicación innecesaria de información. Si existe duplicación por razones de rendimiento, el DBMS se encarga de mantener la consistencia automáticamente.
- Integridad: Asegura la exactitud y validez de los datos almacenados mediante la aplicación de reglas del negocio (restricciones de clave primaria, foránea, unicidad y chequeo).





- Independencia de Datos: Aislamiento entre los programas de aplicación y la estructura física/lógica de los datos, permitiendo modificaciones estructurales sin alterar el código de las aplicaciones.
- Seguridad y Control de Acceso: Protección de los datos contra accesos no autorizados mediante autenticación, autorización y asignación de privilegios.
- Concurrencia: Capacidad de permitir que múltiples usuarios accedan y modifiquen la base de datos simultáneamente sin generar inconsistencias de datos.
- Recuperabilidad (Recuperación ante fallos): Mecanismos para restaurar la base de datos a un estado consistente en caso de fallos de hardware, software o interrupciones de energía (cumpliendo propiedades ACID).

3. Archivos Frente a Bases de Datos: Origen de los Sistemas Gestores

Antes del surgimiento de los Sistemas Gestores de Bases de Datos (SGBD/DBMS), las organizaciones procesaban su información mediante el enfoque tradicional de procesamiento de archivos, donde cada departamento o aplicación desarrollaba y mantenía sus propios archivos independientes. Este modelo presentó severas limitaciones técnicas (Elmasri & Navathe, 2016):

- Redundancia de Datos: Al no existir una administración centralizada, los mismos datos (como la dirección de un cliente) se duplicaban en múltiples archivos de distintos departamentos.
- Inconsistencia de Datos: Como consecuencia de la redundancia, la actualización de un dato en un archivo no se reflejaba automáticamente en los demás, derivando en versiones contradictorias de la verdad.





- **Dependencia Programa-Datos:** La estructura de los registros estaba codificada directamente en las aplicaciones. Cualquier cambio en el formato de un archivo (añadir un campo) requería reescribir y recompilar todos los programas que accedían a él.

Evolución hacia los SGBD: Estas deficiencias críticas dieron origen a los Sistemas Gestores de Bases de Datos. Los SGBD introdujeron el concepto de catálogo centralizado (metadatos) y abstracción, separando la definición de las estructuras de datos de la lógica de procesamiento de las aplicaciones y centralizando el control de integridad, concurrencia y seguridad.

4. Usuarios de una Base de Datos: Roles, Responsabilidades y Permisos

En un entorno empresarial, interactúan diversos perfiles de usuarios con responsabilidades y niveles de acceso diferenciados (Silberschatz et al., 2020):

Rol de Usuario	Responsabilidades Principales	Permisos Típicos en el SGBD
Diseñador de Base de Datos	Identificar los datos a almacenar, elegir estructuras representativas, diseñar los esquemas conceptual y lógico.	Acceso en entornos de desarrollo/pruebas; permisos DDL de creación de prototipos.
Administrador de Base de Datos (DBA)	Gestión de la infraestructura física, seguridad, respaldos, tuning de rendimiento, concesión de accesos y mantenimiento del catálogo.	Privilegios globales de superusuario (`SUPERUSER`, `ALL PRIVILEGES`, gestión de roles).
Programador de Aplicaciones	Desarrollar componentes de software que interactúan con la BD mediante consultas embebidas o APIs (DML/SQL).	Permisos de lectura/escritura (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) en esquemas de desarrollo.





Instituto Politécnico Nacional
Escuela Superior de Computo, ESCOM
Ingeniería en Sistemas Computacionales



Usuario Final	Consumir o actualizar información mediante interfaces gráficas o reportes generados por las aplicaciones.	Privilegios indirectos restringidos a vistas e interfaces a través de roles de aplicación.
----------------------	---	--





5. Ciclo de Vida del Desarrollo de una Base de Datos

El desarrollo sistemático de una base de datos sigue una metodología formal estructurada en etapas secuenciales, donde cada fase produce un entregable auditable (Connolly & Begg, 2015):

Etapas del Ciclo de Vida	Descripción Operativa	Entregable Principal
1. Planificación del Sistema	Definición del alcance del proyecto, recursos necesarios y análisis de factibilidad técnica y económica.	Documento de alcance y plan de proyecto.
2. Definición del Sistema	Especificación de los límites del sistema, usuarios y áreas de aplicación involucradas.	Especificación de límites y reglas del negocio.
3. Análisis de Requisitos	Recolección y análisis de necesidades de información mediante entrevistas y documentación existente.	Documento de Requisitos de Información (SRS).
4. Diseño de la Base de Datos	Elaboración de los modelos Conceptual (Entidad-Relación), Lógico (Relacional) y Físico (Indexación).	Diagrama ER, Esquema Relacional normalizado y Diccionario de Datos.
5. Implementación	Creación física de las tablas, restricciones y disparadores mediante scripts DDL en el SGBD elegido.	Base de datos vacía en el SGBD y scripts DDL.
6. Conversión / Carga de Datos	Migración de datos existentes desde archivos legacy o sistemas previos hacia el nuevo esquema.	Base de datos poblada e informe de migración.





6. Sistema Gestor de Bases de Datos (SGBD): Módulos, Lenguajes y Clasificación

Un SGBD es un complejo sistema de software compuesto por módulos especializados responsables del almacenamiento, procesamiento de consultas y control transaccional.

Módulos Componentes del SGBD

- **Procesador de Consultas:** Incluye el intérprete DDL, el compilador DML y el optimizador de consultas, encargado de transformar sentencias de alto nivel en planes de ejecución eficientes.
- **Gestor de Almacenamiento:** Modulo de bajo nivel que interactúa entre la base de datos almacenada en disco y las peticiones del sistema, incluyendo el gestor de memoria intermedia (buffer manager), gestor de archivos y gestor de índices.
- **Módulo Transaccional:** Compuesto por el gestor de concurrencia y el gestor de recuperación, encargados de mantener las propiedades ACID.

Lenguajes del SGBD

- **DDL (Data Definition Language):** Lenguaje de Definición de Datos (``CREATE``, ``ALTER``, ``DROP``). Define las estructuras del esquema de la base de datos y sus restricciones. Se almacena en el catálogo o diccionario de datos.
- **DML (Data Manipulation Language):** Lenguaje de Manipulación de Datos (``SELECT``, ``INSERT``, ``UPDATE``, ``DELETE``). Permite la consulta y modificación de los datos almacenados.
- **DCL (Data Control Language):** Lenguaje de Control de Datos (``GRANT``, ``REVOKE``). Administra los permisos y derechos de acceso de los usuarios.





- TCL (Transaction Control Language): Lenguaje de Control de Transacciones (`COMMIT`, `ROLLBACK`, `SAVEPOINT`). Gestiona la atomicidad y durabilidad de las unidades de trabajo.

Clasificación de los SGBD

Los SGBD se clasifican según su modelo de datos subyacente: Relacionales (RDBMS) (ej. PostgreSQL, Oracle, MySQL), NoSQL / No Relacionales (orientados a documentos como MongoDB, clave-valor como Redis, grafos como Neo4j), Orientados a Objetos (ODBMS) y Multidimensionales para analítica OLAP.

7. Sistema de Base de Datos: Arquitectura ANSI/SPARC y Estructuras Físicas

La arquitectura estándar para sistemas de bases de datos fue propuesta por el grupo ANSI/X3/SPARC y se estructura en tres niveles de abstracción (Elmasri & Navathe, 2016):

- Nivel Interno (Físico): Describe la estructura de almacenamiento físico de la base de datos, métodos de acceso, organización de archivos en disco e índices.
- Nivel Conceptual (Lógico): Describe la estructura global de toda la base de datos para la comunidad de usuarios. Define entidades, atributos, relaciones y restricciones de integridad, ocultando detalles físicos.
- Nivel Externo (Vistas de Usuario): Describe varias vistas personalizadas para grupos de usuarios específicos, mostrando solo la información relevante para su rol y ocultando el resto.

Esta arquitectura en tres niveles proporciona dos tipos de independencia de datos:





- Independencia Física de Datos: Capacidad de modificar el esquema interno (ej. cambiar índices o reorganizar archivos en disco) sin alterar el esquema conceptual ni los programas de aplicación.
- Independencia Lógica de Datos: Capacidad de modificar el esquema conceptual (ej. añadir campos o tablas) sin alterar los esquemas externos ni los programas de aplicación existentes.

Arquitecturas Centralizada y Cliente-Servidor

En la arquitectura centralizada, todo el procesamiento (SGBD, interfaz y aplicación) ocurre en un único equipo (mainframe/servidor). En la arquitectura cliente-servidor, las funciones se distribuyen: los clientes gestionan la interfaz de usuario y lógica de presentación, mientras que el servidor ejecuta las funciones del SGBD (procesamiento SQL, optimización y concurrencia).

8. Modelos de Datos: Importancia y Clasificación

Un modelo de datos es una colección de conceptos abstractos utilizados para describir la estructura, relaciones, semántica y restricciones de datos en una organización. Su importancia radica en que proporciona un lenguaje común entre desarrolladores y usuarios finales durante la etapa de diseño.

Clasificación por Nivel de Abstracción

- Modelos Conceptuales (Alto Nivel): Ofrecen conceptos cercanos a cómo perciben los datos los usuarios humanos (ej. Modelo Entidad-Relación).
- Modelos Lógicos (Nivel de Representación): Expresan estructuras entendibles por el SGBD pero independientes de la implementación física (ej. Modelo Relacional, Documental).





- Modelos Físicos (Bajo Nivel): Describen cómo se guardan físicamente los datos en el disco (formatos de archivo, ordenamiento de bloques e índices).

Clasificación por Estructura

- Modelo Jerárquico: Estructura en árbol invertido donde un registro padre puede tener varios hijos, pero un hijo solo tiene un padre.
- Modelo en Red: Estructura de grafo donde un registro hijo puede enlazarse con múltiples registros padres.
- Modelo Relacional: Organiza los datos en tablas (relaciones) bidimensionales formadas por filas (tuplas) y columnas (atributos).
- Modelo Orientado a Objetos / Objeto-Relacional: Integra conceptos de clases, métodos, herencia y encapsulamiento con la persistencia de bases de datos.
- Modelos NoSQL (Documental, Clave-Valor, Grafos): Diseñados para alta flexibilidad de esquema, escalabilidad horizontal y manejo de Big Data de alta velocidad.

Ejercicio 4. Estado del Arte: Fichas de Artículos Científicos Arbitrados

A continuación, se presentan las fichas académicas individuales de tres artículos científicos publicados en congresos y revistas internacionales con revisión por pares e indizadas en ACM Digital Library, IEEE Xplore y conferencias de primera línea (SIGMOD, CIDR). Todos los artículos cuentan con DOI, son posteriores a 2018 y son distintos de los clásicos revisados en clase (Codd 1970, Tsichritzis & Klug 1978, Selinger et al. 1979, Chen 1976 y Härder & Reuter 1983).

Ficha Técnica 1: Kraska et al. (2018) – Índices Aprendidos con IA





1. Cita Completa en Formato APA 7ª Edición con DOI: Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Neokleous, N. (2018). The case for learned index structures. In Proceedings of the 2018 International Conference on Management of Data (ACM SIGMOD '18), pp. 489–504.
<https://doi.org/10.1145/3183713.3196909>
2. Problema que Aborda el Artículo: Plantea que las estructuras de datos de indexación tradicionales (como B-Trees, Hash Tables y Bloom Filters) tratan las claves de forma agnóstica sin aprovechar los patrones de distribución subyacentes de los datos. Esto resulta en un consumo excesivo de memoria RAM y degradación de velocidad de búsqueda en bases de datos a escala de petabytes.
3. Método o Propuesta de los Autores: Demuestran matemáticamente que los B-Trees pueden reestructurarse como modelos de aprendizaje automático (modelos de regresión acumulativa / redes neuronales). Proponen la arquitectura 'Learned Index Structures' y el algoritmo 'Recursive Model Index' (RMI), que entrena modelos simples en jerarquía para predecir la posición exacta de un registro en disco o memoria.
4. Resultado Principal que Reportan: Demuestran empíricamente en datasets del mundo real (datos genómicos, logísticos y de mapas) que los índices aprendidos superan a los B-Trees tradicionales, logrando reducciones del consumo de memoria de hasta un 70% a 90% y aceleraciones de hasta 70% en el tiempo de acceso de lectura.
5. Relación Explícita con la Unidad Temática I: Se relaciona directamente con el Tema 6 (Módulos Componentes del SGBD - Gestor de Almacenamiento e Índices) y el Tema 7 (Arquitectura Nivel Físico/Interno), al transformar radicalmente los métodos físicos de organización de archivos y estructuras de acceso en disco.
6. Aporte para el Proyecto del Curso: Nos aporta una visión moderna sobre la optimización del diseño físico y tuning de bases de datos. Al diseñar nuestro





proyecto, nos permite comprender cómo el SGBD gestiona los índices internamente en memoria RAM y por qué es crítico evitar la sobre-indexación de tablas.

Ficha Técnica 2: Armbrust et al. (2021) – Arquitectura Lakehouse

1. Cita Completa en Formato APA 7ª Edición con DOI: Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR '21), Paper 17. <https://doi.org/10.1145/3432432.3432435>
2. Problema que Aborda el Artículo: Analiza la ineficiencia de las arquitecturas empresariales modernas de dos capas que separan los Data Lakes (para datos no estructurados) y los Data Warehouses (para analítica SQL). Esta dualidad genera duplicación masiva de datos, falta de consistencia ACID, elevados costos de infraestructura y falta de frescura en la información.
3. Método o Propuesta de los Autores: Proponen formalmente la arquitectura 'Lakehouse', la cual implementa una capa de almacenamiento transaccional y metadata abierta (como Delta Lake) directamente sobre almacenamiento de objetos de bajo costo (ej. S3). Esto permite ejecutar transacciones ACID, versionado de datos ('time travel') y aislamiento de concurrencia sobre archivos estructurados y no estructurados.
4. Resultado Principal que Reportan: Demuestran que Lakehouse elimina la necesidad de procesos ETL complejos entre Data Lakes y Warehouses, ofreciendo rendimiento de consultas SQL comparable o superior a los Data Warehouses relacionales tradicionales con un costo hasta 10 veces menor y garantizando consistencia ACID completa.
5. Relación Explícita con la Unidad Temática I: Se relaciona con el Tema 2 (Características: Integración, Persistencia, Integridad y Concurrencia) y el





Tema 7 (Arquitecturas de Procesamiento Cliente-Servidor / Distribuidas), mostrando cómo mantener las propiedades transaccionales clásicas en infraestructuras masivas en la nube.

6. Aporte para el Proyecto del Curso: Nos ayuda a fundamentar la elección del motor de base de datos y la arquitectura de almacenamiento para nuestro proyecto, enseñándonos cómo estructurar datos heterogéneos garantizando consistencia sin sacrificar la escalabilidad del almacenamiento.

Ficha Técnica 3: Neumann & Freitag (2020) – Motores SQL In-Memory con Umbra

1. Cita Completa en Formato APA 7ª Edición con DOI: Neumann, T., & Freitag, M. (2020). Umbra: A disk-based system with in-memory performance. In Proceedings of the 10th Conference on Innovative Data Systems Research (CIDR '20), Paper 29. <https://doi.org/10.1145/3318464.3389712>
2. Problema que Aborda el Artículo: Aborda el dilema entre las bases de datos puramente en memoria (In-Memory DBMS), que están limitadas por la capacidad física de la RAM, y las bases de datos orientadas a disco tradicionales, que sufren graves cuellos de botella de E/S al procesar analítica analítica compleja.
3. Método o Propuesta de los Autores: Diseñan el sistema gestor 'Umbra', que utiliza un gestor de memoria intermedia de bajo overhead basado en páginas de tamaño variable y compilación JIT (Just-In-Time) de consultas SQL a código máquina nativo mediante LLVM, combinando la velocidad de procesamiento en RAM con la capacidad de persistencia ilimitada de los discos NVMe.
4. Resultado Principal que Reportan: Reportan que Umbra alcanza una velocidad de procesamiento de consultas SQL idéntica a los SGBD In-





Memory puros cuando los datos caben en RAM, y mantiene un rendimiento degradado de manera suave y predecible cuando el volumen de datos excede la memoria principal y requiere acceso a disco.

5. Relación Explícita con la Unidad Temática I: Se conecta directamente con el Tema 6 (Módulos Componentes del SGBD: Compilador DML, Optimizador y Buffer Manager) y el Tema 3 (Archivos vs. Bases de Datos), demostrando la sofisticación de los gestores modernos para superar las limitaciones de E/S de archivos.
6. Aporte para el Proyecto del Curso: Nos proporciona criterios técnicos claros para comprender la gestión de buffers de memoria y la importancia de escribir consultas SQL optimizadas (DML) que faciliten la compilación y ejecución eficiente por parte del optimizador del SGBD.

Cierre del Ejercicio: Análisis Comparativo del Estado del Arte y Problema Abierto

Elementos en Común: A pesar de abordar distintas capas de la arquitectura de datos, las tres investigaciones estudiadas (Kraska et al., 2018; Armbrust et al., 2021; Neumann & Freitag, 2020) comparten el objetivo central de rediseñar los componentes clásicos de los SGBD para superar los cuellos de botella impuestos por la escala masiva de datos (Big Data) y el hardware moderno. Todas las propuestas coinciden en la necesidad de preservar las garantías fundamentales de las bases de datos (consistencia, persistencia e integridad) mientras aprovechan avances contemporáneos como la compilación de código en tiempo





de ejecución, el almacenamiento masivo distribuido en la nube y los modelos de aprendizaje automático.

Puntos de Diferencia: Las diferencias radican en su nivel de abstracción y foco arquitectónico. Kraska et al. operan a nivel micro de estructuras de datos físicas, reemplazando árboles de búsqueda por modelos predictivos. Neumann & Freitag se enfocan en la capa de procesamiento del SGBD, optimizando la gestión de memoria RAM/disco y la compilación DML. Por su parte, Armbrust et al. adoptan una perspectiva de macro-arquitectura de sistemas distribuidos, unificando repositorios heterogéneos bajo un estándar transaccional unificado en la nube.

Identificación de un Problema Abierto: A partir de la lectura crítica de esta literatura, se identifica como problema abierto la Auto-adaptabilidad Dinámica y Verificación de Garantías ACID en Índices Aprendidos bajo Transacciones Concurrenciales de Alta Frecuencia. Si bien los índices aprendidos (Kraska et al.) funcionan de forma sobresaliente en cargas de trabajo de solo lectura (OLAP), su reentrenamiento en entornos altamente transaccionales (OLTP) con miles de escrituras concorrentes por segundo genera un costo computacional prohibitivo. Resolver la degradación del índice durante modificaciones DML concurrentes sin perder el aislamiento de transacciones sigue siendo una frontera abierta en la investigación de bases de datos autónomas.





Referencias Bibliográficas

Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR '21), Paper 17. <https://doi.org/10.1145/3432432.3432435>

Connolly, T. M., & Begg, C. E. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed.). Pearson Education.

Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Neokleous, N. (2018). The case for learned index structures. In Proceedings of the 2018 International Conference on Management of Data (ACM SIGMOD '18), pp. 489–504.

<https://doi.org/10.1145/3183713.3196909>

Neumann, T., & Freitag, M. (2020). Umbra: A disk-based system with in-memory performance. In Proceedings of the 10th Conference on Innovative Data Systems Research (CIDR '20), Paper 29. <https://doi.org/10.1145/3318464.3389712>

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

